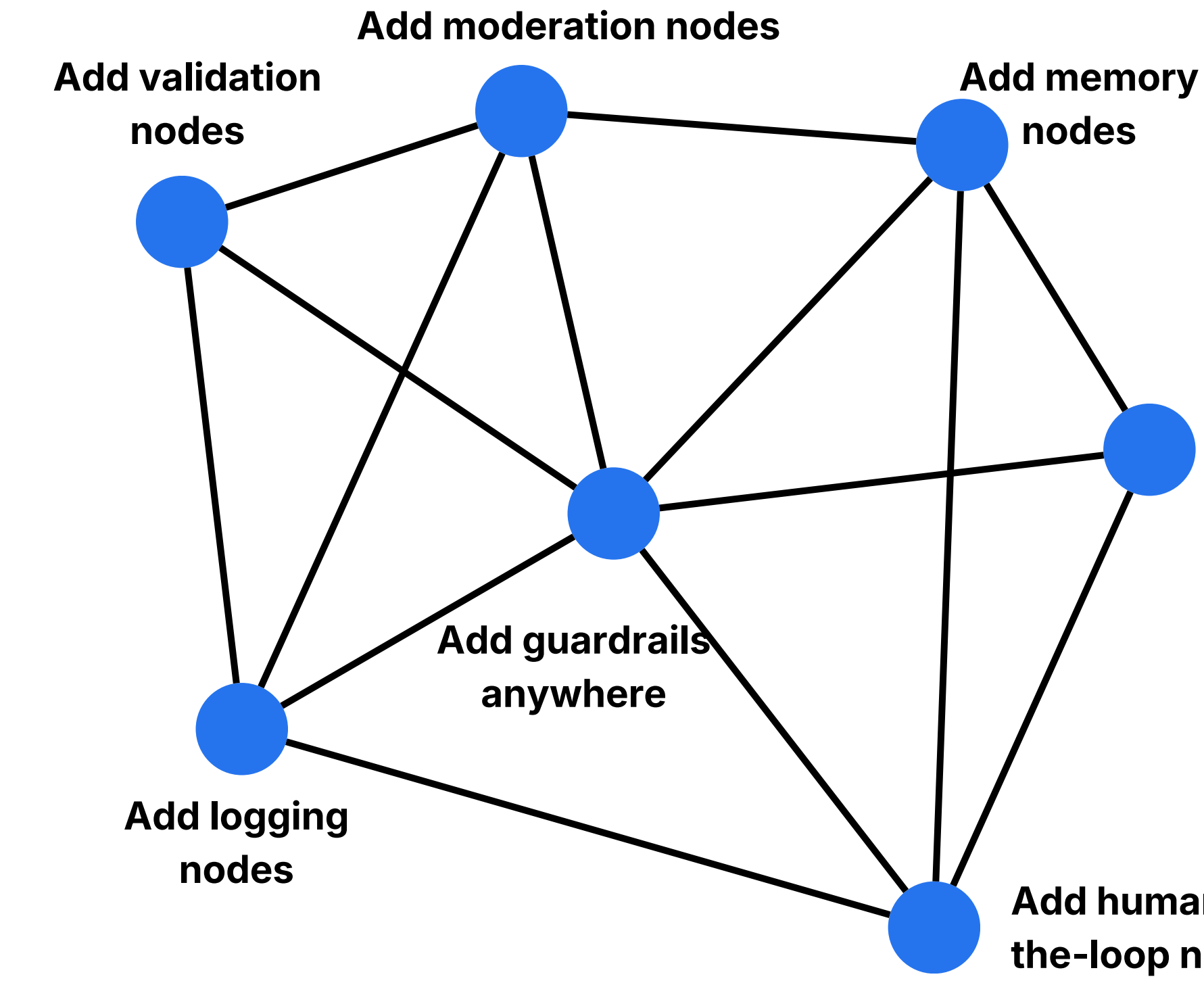
Analytics Vidhya

LangChain vs LangGraph



LangChain is a **high-level interface** for **building agents**.

LangGraph is the **low-level runtime** that actually **executes them**.

If you’re building LLM agents today, you’re almost certainly interacting with LangChain in some form. But under the hood, there’s a newer layer doing the real orchestration work: **LangGraph**.

Here’s the key idea most people miss:
LangChain agents are built on top of LangGraph.

Once you understand that relationship, the differences become much clearer.

Let’s break it down properly.

LangChain is a **high-level interface** for **building agents**.

LangGraph is the **low-level runtime** that actually **executes them**.

🦜🔗 **LangChain** - - - - - - - - - - ➤ makes things **easy** 🫠

 LangGraph - - - - - - - - - - ➤ makes things **powerful** ⚡

Analytics Vidhya

`create_agent` in LangChain is a **LangGraph ReAct Agent**

```python
from langchain.agents import create_agent

agent = create_agent(
    model="openai:gpt-5.1 mini",
    tools=[search_tool],
)
```

LangChain is **constructing a ReAct** agent using **LangGraph**.

Specifically:

- It creates a LangGraph state machine. Adds nodes like:
  - Model node
  - Tool node
  - Decision logic node
- Connects them in a loop
- Executes until stopping condition

So this:

```python
create_agent(...)
```

is essentially a **prebuilt LangGraph workflow**.

LangChain - - - - - - - - - - → convenience layer

LangGraph - - - - - - - - - - → execution layer

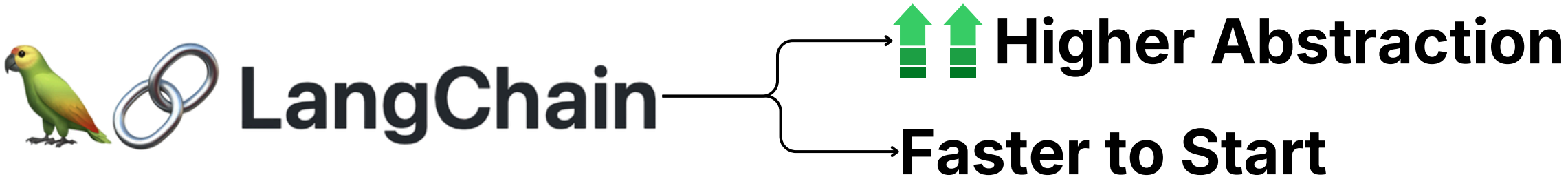
Core Architectural Difference

Here’s the cleanest mental model:

Layer	Responsibility
🦜🔗 LangChain	Agent interface, abstraction, developer experience
LangGraph	Agent execution engine, state, orchestration

🦜🔗 LangChain - - - - - - - - - - ➤ is the engine.

LangGraph- - - - - - - - - - ➤ is the dashboard.



LangChain is designed for **developer productivity**.

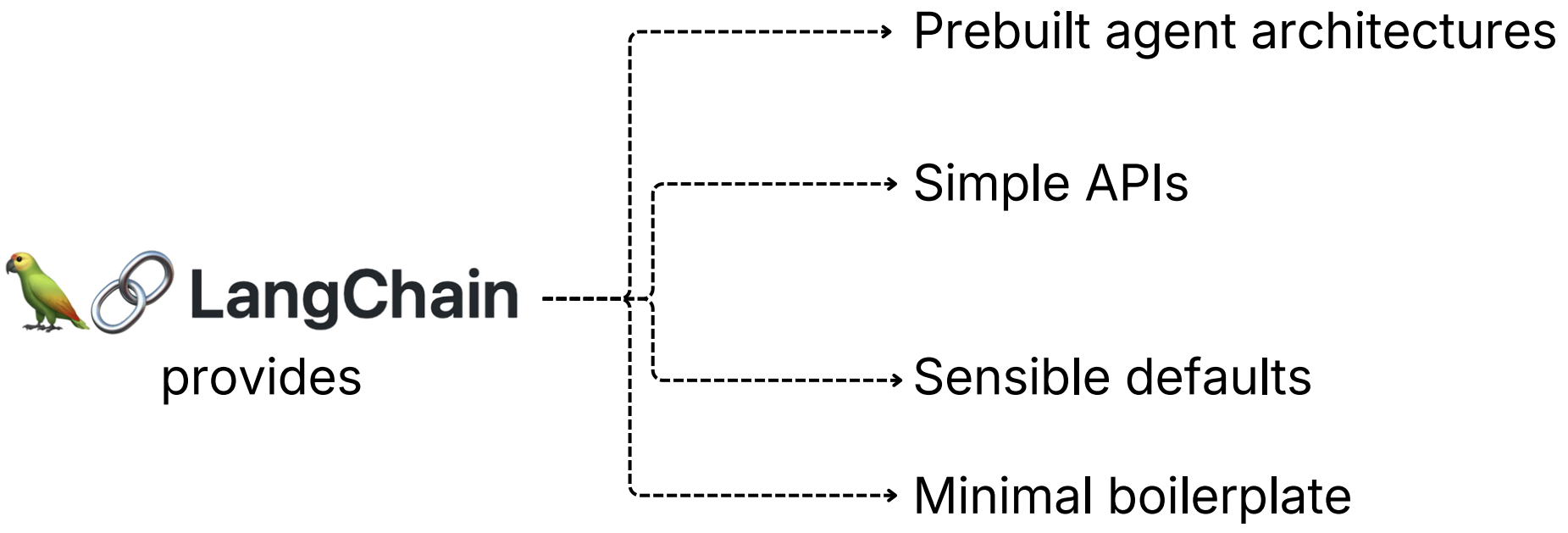
It hides complexity so you can build agents quickly.

```
agent = create_agent(
    model="gpt-4o",
    tools=[calculator, search]
)
```

You don’t need to worry about:

- State machines
- Execution graphs
- Transitions
- Loop control

LangChain handles everything.



Because LangChain abstracts things away, you lose fine-grained control. You cannot easily:

- Insert custom execution steps
- Modify internal decision loops
- Add advanced guardrails at arbitrary points
- Control execution flow precisely

You get convenience.

But less flexibility

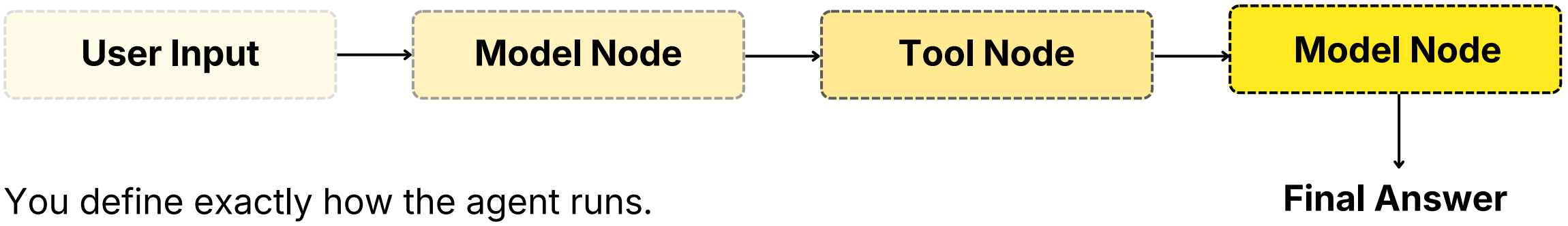
LangGraph → ⬇⬇Lower Abstraction

→ Full Control

LangGraph exposes the actual execution model.

- You build agents as graphs.
- Nodes represent steps.
- Edges represent transitions.

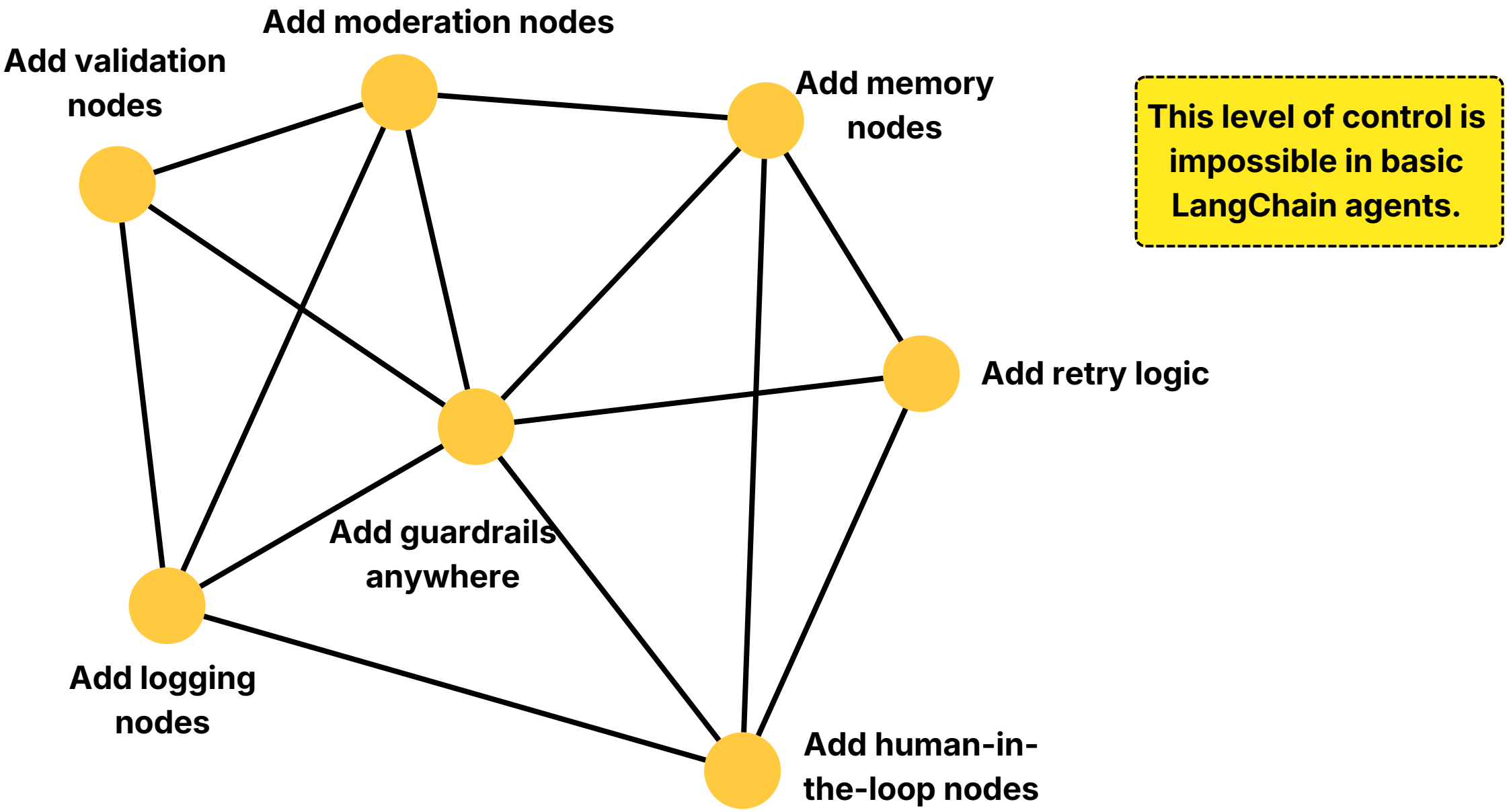
Example structure:



You define exactly how the agent runs.

Why LangGraph is More Powerful

Because you control the graph, you can:



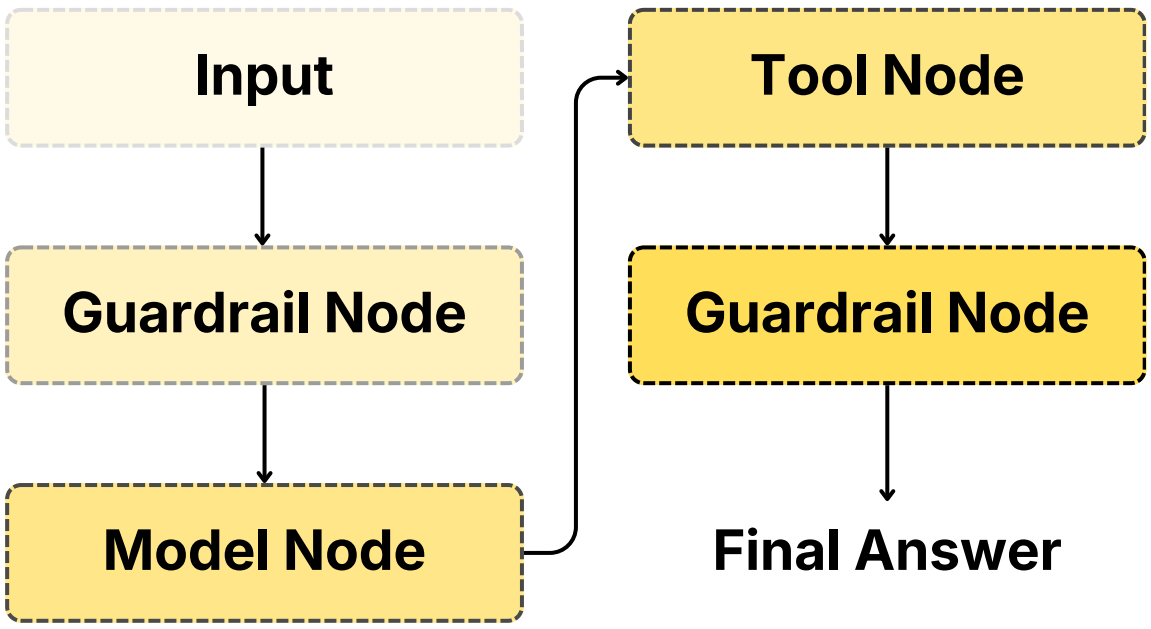
Middleware in LangChain is Actually Adding LangGraph Nodes

This is another important concept.

LangChain middleware works by inserting additional nodes into the LangGraph execution graph. For example, guardrails middleware:

```
agent = create_agent(
    model=model,
    tools=tools,
    middleware=[guardrail]
)
```

Internally, this becomes:



Middleware is not just a wrapper.

It modifies the underlying **LangGraph graph**.

This is why **LangGraph** is the **true execution layer**.

Why LangGraph Has a Steeper Learning Curve

LangGraph requires understanding:

- State management
- Graph execution
- Nodes
- Edges
- Execution cycles
- Agent loops

Example LangGraph skeleton:

```
from langgraph.graph import StateGraph

builder = StateGraph(State)

builder.add_node("model", call_model)
builder.add_node("tools", call_tools)

builder.add_edge("model", "tools")
builder.add_edge("tools", "model")

graph = builder.compile()
```

This gives full control. But requires more work.

Direct Comparison

Here’s the honest comparison.

Feature	LangChain	LangGraph
Abstraction level	High	Low
Ease of use	Very Easy	Harder
Learning curve	Low	Steep
Control	Limited	Full
Custom workflows	Difficult	Easy
Guardrails	Middleware only	Native
Debugging	Limited Visibility	Full Visibility
Production reliability	Good	Excellent
Flexibility	Moderate	Maximum